

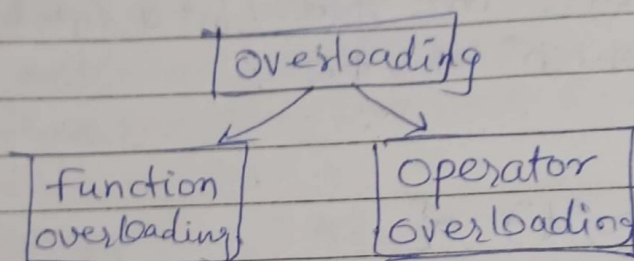
Overloading +

- * If we create two or more members having the same name but different in number or type of parameter, It is known as C++ overloading.

In C++, we can overload :

- methods,
- constructors and
- Indexed properties.

It is bcoz these members have parameters only.



* C++ function overloading +

Function overloading is defined as the process of having two or more functions with the same name, but different in parameters is known as function overloading.

In function overloading, the function is redefined by using either different types of arguments or diff. number of arguments.

Its advantage is that it increases the readability of the prg, bcoz you don't need to use different names for the same action.

C++ function overloading ex.

(a) Prg. of function overloading when number of arguments vary.

```
#include <iostream>
using namespace std;
class Cal {
public:
    static int add (int a, int b)
    {
        return a+b;
    }
    static int add (int a, int b, int c)
    {
        return a+b+c;
    }
};

int main (void) {
    Cal C; // class object declaration
    cout << C.add (10, 20) << endl;
    cout << C.add (12, 20, 23) ;
    return 0;
}
```

op → 30
55

(b) Prg. of function overloading with different types of arguments.

```
#include <iostream>
using namespace std;
int mul (int, int);
float mul (float, int);
```

```
int mul (int a, int b)
{
```

```
    return a * b;
```

```
}
```

```
float mul (double x, int y)
{
```

```
    return x * y;
```

```
}
```

```
int main()
```

```
{
```

```
    int r1 = mul (6, 7);
```

```
    float r2 = mul (0.2, 3);
```

```
    std::cout << "r1 is: " << r1 << std::endl;
```

```
    std::cout << "r2 is: " << r2 << std::endl;
```

```
    return 0;
```

```
}
```

op

r1 is: 42

r2 is: 0.6

(c) Prog. of function overloading with diff. sequence of arguments.

```
#include <iostream>
```

```
using namespace std;
```

```
void add (int a, double b)
```

```
{
```

```
    cout << "sum = " << (a+b);
```

```
}
```

```
void add (double a, int b)
```

```
{
```

```
    cout << endl << "sum = " << (a+b);
```

```
}
```

// driver code

```

int main ()
{
    add (10, 2.5);
    add (5.5, 6);
    return 0;
}

```

op sum = 12.5
 sum = 11.5

function Overloading & ambiguity +

* When a compiler is unable to decide which function is to be invoked among the overloaded function, this situation is known as function overloading.

* When the compiler shows the ambiguity error, the compiler does not run the program.

causes of fun. overloading + causes of Ambiguity +

- 1) Type conversion
- 2) function with default arguments
- 3) function with pass by reference.

1) Type conversion +

```

#include <iostream>
using namespace std;
void fun(int);
void fun(float);
void fun(int i) i

```

```

{
    std::cout << "Value of i is : " << i << std::endl;
}
void fun(float j)
{
    std::cout << "Value of j is: " << j << std::endl;
}
int main()
{
    fun(12);
    fun(1.2);
    return 0;
}

```

The above eg. shows an error
 "call of overloaded fun(double) is ambiguous".
 The fun(10) will call the first function.
 The fun(1.2) calls the second function
 according to our prediction.

But, this does not refer to any function as in C++ all the floating point constants are treated as double not as float.
 * If we replace float to double, the prog. works. Therefore, this is a type conversion from float to double.

2) Functions with default arguments +

```

#include <iostream>
using namespace std;
void fun(int);
void fun(int, int);
void fun(int i)
{
    std::cout << "Value of i is: " << i << std::endl;
}

```

void fun (int a, int b=9)

```
{
    std::cout << "value of a is: " <<
        a << std::endl;
    std::cout << "value of b is: " <<
        b << std::endl;
}
```

```
int main()
{
```

```
* u    fun(12);
    c
    c    return 0;
    }
```

* The above eg. shows an error "call of overloaded 'fun(int)' is ambiguous". The fun (int a, int b=9) can be called in two ways :
 first is by calling the function with one argument ie. f(12) &
 another way is calling the function with two arguments ie. f(4,5).

1)
 2)
 3) The function fun(int i) is invoked with one argument. Therefore the compiler could not be able to select among fun(int i) and fun (int a, int b=5)

1) Ty

```
# incl
using
void
```

```
void
void
```

3) function with pass by reference

```
# include <iostream>
using namespace std;
void fun(int);
void fun (int &);
int main()
```

```
{
```

```

int a=10 ;
fun(a) ; //error, which f() &
return 0 ;
}

void fun (int x)
{
    std::cout << "Value of x is ; " << x <<
        std::endl ;
}

void fun (int &b)
{
    std::cout << "Value of b is ; " << b << std::endl ;
}

```

The above eg. shows an error.

"call of overloaded 'fun (int&)' is ambiguous". The first function takes one integer argument & the second function takes a reference parameter as an argument.

In this case, the compiler does not know which function takes by the user as there is no syntactical diff. betⁿ the fun (int) & fun (int&).

Constructor Functions +

C++ Constructor +

- * constructor is a special method which is invoked automatically at the time of object creation.
- * It is used to initialize the data members of new object generally.
- * Constructors don't have a return value & not have return type.

There can be 2 types of constructor :-

- 1) Default constructor
- 2) Parameterized constructor

➤ C++ Default constructor :-

A constructor which has no argument is known as default constructor.

It is invoked at the time of creating object.

```
eg. #include <iostream>
using namespace std;
class Employee
{
public :
    Employee ()
    {
        cout << "Default Constructor Invoked "
              << endl;
    }
};

int main (void)
{
    Employee e1; // creating an object of Employee
    Employee e2;
    return 0;
}
```

O/P :-

Default constructor Invoked
Default constructor Invoked

2) C++ Parameterized Constructor

A constructor which has parameters is called parameterized constructor.

It is used to provide diff. values to distinct objects.

eg.

```
#include <iostream>
using namespace std;
class Employee {
public:
    int id; // data member (instance variable)
    string name; // data member (instance variable)
    float salary;
    Employee (int i, string n, float s)
    {
        id = i;
        name = n;
        salary = s;
    }
    void display ()
    {
        cout << id << " " << name << " " << salary << endl;
    }
};

int main (void) {
    Employee e1 = Employee (101, "Om", 89000);
    // creating an object of Employee
    Employee e2 = Employee (102, "Jay", 59000);
    e1.display ();
    e2.display ();
    return 0;
}
```

Output → 101 Om 89000
102 Jay 59000

What is a copy constructor +

A member function known as a copy constructor which initializes an item using another object from the same class.

* Every time we specify one or more non-default constructors (with parameters) for a class, we also need to include a default constructor (without parameters) as the compiler won't supply one in this circumstance.

The best practice is to always declare a default constructor, even though it is not required.

A reference to an object belonging to the same class is required by the copy constructor.

```
Sample (Sample &t)
{
    id = t.id;
}
```

Localizing variables +

A local variable is defined as whose initial value is set and consumed within a function or method.

These variables gets life only when the function where the variable exists is executed & gets destroyed automatically when program control passes to next function.

Local variables +

variables defined within a function or block are said to be local to the function.

eg.

```
#include <iostream>
using namespace std;
void func()
{
    int age = 18; // this variable is
                  // local to the funn func() &
                  // cannot be accessed outside
                  // this function
    cout << age;
}
int main()
{
    cout << "Age is: " << age;
    return 0;
}
cout << "Age is: " ;
```

op →

Error: age was not declared in this scope.

Rectified prg. +

written by pencil.

Rec Global Variables

* As the name suggests, Global variables can be accessed from any part of the program.

* They are available throughout the life time of a prog.

* They are declared at the top of the prog. outside all the funⁿ & can be accessed from any part.

eg.

```
#include <iostream>
```

```
using namespace std;
```

```
int global = 5; // global variable
```

```
void display ()
```

```
{
```

```
    cout << global << endl; // global
```

```
} // variable accessed from  
    within a function.
```

```
int main ()
```

```
{
```

```
    display ();
```

```
    global = 10; // changing value of  
    global variable from  
    main function
```

```
    display ();
```

```
}
```

opt

5

10

Q. What if there exists a local variable with the same name as that of global variable inside a function?

Ans.

Usually when two variables with same name are defined then the compiler produces a compile time error. But if the variables are defined in diff. scopes then the compiler allows it.

Whenever there is a local variable defined with same name as that of a global variable then compiler will give precedence to the local variable.

Q. How to access a global variable when there is a local variable with same name?

Ans.

using scope resolution operator.

eg.

```
#include <iostream>
using namespace std;
int x = 0; // Global x;
int main ()
{
    int x = 10; // local x;
    cout << "Value of global x is " << ::x;
    cout << "Value of local x is " << x;
    return 0;
}
```

o/p

value of global x is 0.
value of local x is 10.

Operator overloading

* Operator overloading is a compile-time polymorphism.

* main idea behind operator overloading is to use c++ operators with class variables or class objects.

Redefining the meaning of operators really does not change their original meaning instead they have been given additional meaning along with their existing ones.

eg

```
#include <iostream>
```

```
using namespace std;
```

```
class Complex {
```

```
private :
```

```
    int real , imag;
```

```
public :
```

```
    Complex (int r=0, int i=0)
```

```
{
```

```
    real = r;
```

```
    imag = i;
```

```
}
```

```
Complex operator + (Complex const & obj)
```

```
// This is automatically called when
```

```
// '+' is used with between two
```

```
// Complex objects.
```

```
{
```

```
    Complex res;
```

```
    res.real = real + obj.real;
```

```
    res.imag = imag + obj.imag;
```

```
    return res;
```

```
}
```

```

void print ()
{
    cout << real << " + i " << imag << '\n';
}

int main ()
{
    Complex c1 (10, 5), c2 (2, 4);
    Complex c3 = c1 + c2;
    c3.print ();
}

```

O/p :-

12 + i9

Following operators cannot be overloaded +

- 1) sizeof operator
- 2) typeid operator
- 3) scope resolution (::) operator
- 4) class member access operators
(. (dot), .* (pointer to member operator))
- 5) Ternary or conditional (? :) operator

Operators that can be overloaded +

- 1) Unary operators
- 2) Binary operators
- 3) Special Operators ([], (), etc).

1) sizeof +

This returns the size of the object or datatype entered as the operand. This is evaluated by the compiler and cannot be evaluated during runtime. The proper incrementing of a pointer in an array of obj. relies on the sizeof operator implicitly.

2) typeid +

This provides a C++ prog. with the ability to recover the actually derived type of the object referred to by a pointer or reference.

For this operator, the whole point is to uniquely identify a type.

* If we want to make a user-defined type 'look' like another type, polymorphism can be used but the meaning of the typeid operator must remain unaltered.

3) Scope resolution (::) +

This helps identify & specify the context to which an identifier refers by specifying a namespace.

It is completely evaluated at runtime & works on names rather than values.

4) Class member access operators

(. (dot), .* (pointer to member operator)) ; the importance of and implicit use of class member access operators can be understood through the

5) Ternary or conditional (? :) +

It is a shorthand representation of an if-else statement

This ternary operator cannot be overloaded.

Rules of Operator overloading +

- 1) Only ~~ex~~ existing operators can be overloaded. New operators cannot be created.
- 2) The overloaded operator must have at least one operand that is of user-defined class object.
- 3) We cannot change the basic meaning of an operator.
- 4) Overloaded operators follow the syntax rules of the original operator.
- 5) We cannot use friend function to overload certain operators.

Operators that can be overloaded +

Overloading the Unary Operators +

- * Unary operators operate on single operand.
- * In the unary operator function, no arguments should be passed.
- * It works only with one class object.
- * Consider the unary minus (-) operator. This operator changes the sign of the operand.

Example 1 + C++ prog. for unary operator overloading.

```

#include <iostream>
using namespace std;
class SYIT
{
    int x, y;
public:
    SYIT()
    {
        x = 10;
        y = -20;
    }
    void operator -(); // overloading unary operator
    void display()
    {
        cout << "x = " << x << "\n";
        cout << "y = " << y << "\n";
    }
};

void SYIT :: Operator -() // scope resolution operator.
{
    x = -x;
    y = -y;
}

void main()
{
    SYIT s;
    s.display();
    -s; // use unary opr. by single operand
    s.display();
}

```

opr

```

x = 10
y = -20
x = -10
y = 20

```

Example 2 + C++ prg. for overloading
Increment operator for displayed
no. between 1 to 10.

```
#include <iostream>
using namespace std;
class series
{
    int i, no;
public:
    void operator ++() ;
};

void series:: Operator ++()
{
    for( i=1 ; i<=10 ; i++)
        cout << "\n" << i ;
}

void main ()
{
    series s ;
    ++ s ;
}
```

opt 1 2 3 4 5 6 7 8 9 10
 2 3 4 5 6 7 8 9 10 11.

erand

constructor

eg. A constructor is declared & defined as follows -

```
class student
{
    int m, n;
    public :
        student(void);    // constructor declared
};
```

```
class → Student :: Student (Void) // constructor define
name      {
            m=0;
            n=0;
        }
```

The constructor functions have some special characteristics. These are -

- They should be declared in the public section.
- They are invoked automatically when the objects are created.
- They do not have return types, so they not return values.
- They cannot be inherited through derived class.
- They can have default arguments.
- constructors cannot be virtual.
- we cannot refer to their addresses.

Overloading Binary operators using member function

- * Binary operators operate on 2 operands.
- * In overloading binary operators, the left-hand operand is used to invoke the operator function & right-hand operand is passed as an argument.

Example C++ prg. to find out factorial of given number using '*' operator overloading.

```
#include <iostream>
using namespace std;
class factorial
{
    int i;
    int fact;
public:
    factorial()
    {
        i = 0;
        fact = 1;
    }
    int operator * (int n);
};
int factorial::operator * (int n)
{
    for (i = 1; i <= n; i++)
        fact = fact * i;
    return fact;
}
```

1) Default

Page No.

Date : / /

Ex. write a c++ prg. to display a series 1, 2, ..., 10 using the constructor.

```
#include <iostream>
using namespace std;
class series
{
    int cnt, no;
public:
    series(); // default constructor
    void display();
};

series::series() // constructor definition
{
    cnt = 0;
    no = 10;
}

void series::display()
{
    for (cnt = 1; cnt <= 10; cnt++)
        cout << cnt << " ";
}

void main()
{
    series s;
    s.display();
}
```

3) Parameterized

Ex. C++ prg. display Roll no. & Percentage using parameterized constructor.

```
#include <iostream>
using namespace std;
```

```
class student
```

```
{
```

```
    int Roll no ;
```

```
    float percentage ;
```

```
public :
```

```
    Student (int no, float per) ;
```

```
    void display ()
```

```
{
```

```
    cout << " Roll no " << Roll no << "\n";
```

```
    cout << " percentage " << percentage << "\n";
```

```
}
```

```
};
```

```
Student :: Student (int no, float per)
```

```
{
```

```
    Roll no = no ;
```

```
    Percentage = per ;
```

```
}
```

```
void main ()
```

```
{
```

```
    student s1 (1, 80.10) ;
```

```
    student s2 = student (2, 75) ;
```

```
    cout << "\n object 1 \n";
```

```
    s1.display ();
```

```
    cout << "\n object 2 \n";
```

```
    s2.display ();
```

```
}
```

opt

object 1

Roll no : 1

per : 80.10

object 2

Roll no : 2

per : 75

```
void main()
```

```
{
```

```
    factorial f ;
```

```
    int n, fact ;
```

```
    cout << "Enter the number " ;
```

```
    cin >> n ;
```

```
    fact = f * n ;
```

```
    cout << "factorial of given number is: "
```

```
        << fact ;
```

```
    return 0 ;
```

```
}
```

Operator overloading Using friend function +

* friend function can also be used with operator overloaded functions.

* friend requires only one argument for overloading unary operators & (2) binary operators for overloading binary operators.

Example 1 + C++ prg to perform complex number arithmetic using friend function operator overloading.

Copy constructor

follo. stmt will initialize string 2 using string 1;

```
class string string2 (string1);
```

Ex. prg. to display city name using copy constructor

```
#include <iostream>
```

```
#include <cstring>
```

```
using namespace std;
```

```
class city
```

```
{
```

```
    char cname [15];
```

```
public :
```

```
    city (char c[15]) // parameterised const.
```

```
{
```

```
        strcpy (cname, c);
```

```
}
```

copy → city (city s)

Constructor {

```
        strcpy (cname, s.cname);
```

```
}
```

```
void display ()
```

```
{
```

```
    cout << "city name = " << cname;
```

```
}
```

```
};
```

```
void main ()
```

```
{
```

```
    city s1 ("pune"); // call to Para const.
```

```
    city s2 (s1); // call to copy const
```

```
    cout << "object s1: ";
```

```
    s1.display ();
```

```
    cout << "object s2: ";
```

```
    s2.display ();
```

```
}
```

o/p + Object s1: City Name = Pune
Object s2: City Name = Pune

```
#include <iostream.h>
```

```
using namespace std;
```

```
class complex
```

```
{
```

```
    float real , imag ;
```

```
    public :
```

```
        complex () { }
```

```
        complex (float r, float i )
```

```
        {
```

```
            real = r ,
```

```
            imag = i ;
```

```
        }
```

```
complex operator + (Complex) ;
```

```
friend Complex Operator - ( complex, complex);
```

```
void display ()
```

```
{
```

```
    cout << real << "+" << imag << "i" << "\n" ;
```

```
}
```

```
};
```

without friend function

```
complex complex :: operator + ( complex c)
```

```
{
```

```
    complex temp ;
```

```
    temp.real = c.real + real ;
```

```
    temp.imag = c.imag + imag ;
```

```
    return temp ;
```

```
}
```

with friend function

```
complex operator - ( complex c1 , complex c2)
```

```
{
```

```
    complex temp ;
```

```
    temp.real = c1.real - c2.real ;
```

```
    temp.imag = c1.imag - c2.imag ;
```

```
    return temp ;
```

```
}
```

```
void main ()
```

```
{  
    complex c1 (10.5, 30.5), c2 (9.5, 14.5), c3;
```

```
    cout << "\n first complex no is: ";  
    c1.display();
```

```
    cout << "\n Second complex no is: ";  
    c2.display();
```

```
    cout << "\n complex no. addition is = ";  
    c3 = c1 + c2;  
    c3.display();
```

```
    cout << "\n Complex number subtraction  
            is = ";
```

```
    c3 = c2 - c1;  
    c3.display();
```

```
}
```

Type Conversion

- * When constant & variables of diff. types are mixed in an expression, compiler applies automatic type conversion to the operand as per certain rules.
- * The type of data to the right of an assignment operator is automatically converted to the type of the variable on the left.

```
int m ;
```

```
Float X = 3.14 ;
```

```
m = X ;
```

- convert X to an integer before its value is assigned to m.
- Thus the fractional part is truncated.

- 1) Basic type to class type
- 2) class type to basic type
- 3) one class type to another class type

1)

The conversion from basic to class type is easily carried out. It is automatically done by the compiler with the help of in-built routine or by applying type casting.

Here the left hand operand of = sign is always class type & the right hand operand is always basic type.